

Dataset 7: Uber Play Store Reviews

Decision Support System for Sentiment Classification

DSAI4205 - Big Data Analytics

NAME:	STUDENT ID:
Ma Jiawei	23102792D
Zhang Xulu	24113522D
Xu Wenqi	22098103D
Sun Qi	23099458D
Zou Lanxin	23097165D
Chen Hongli	23097738D

Hong Kong Polytechnic University
April 2026

Contents

1	Exploratory Data Analysis (EDA)	2
1.1	Data Granularity	2
1.2	Dataset Overview & Missing Values	2
1.3	Target Variable: Rating Distribution	2
1.4	Text Characteristics & Quality Diagnostics	3
1.5	Outliers	4
1.5.1	Thumbs-Up Outliers	4
1.5.2	Review-Length Outliers	4
2	Data Cleaning and Preprocessing	5
2.1	Feature Selection and Missing Value Handling	5
2.2	Text Normalization and Strict Data Filtering	5
2.3	Sentiment Label Mapping: From 5-Star to 3-Class	6
2.4	Data Splitting and Balancing via NLLB Back-Translation	6
2.5	Final Dataset Composition	6
3	Phase 3i: Baseline Model Analysis	6
3.1	Objective and Data Preparation	6
3.2	Methodology: TF-IDF + Logistic Regression	7
3.3	Baseline Performance	7
3.4	Limitations and Error Analysis	7
3.5	Strategic Transition: From Traditional to Contextual	8
4	Phase 3ii: Advanced Modeling	8
4.1	Model A: GloVe + BiLSTM	8
4.2	Model B: BERT + XGBoost	9
4.3	Model C: BERT + Random Forest	9
4.4	Model D: FastText + SVM	10
4.5	Model E: FastText + Random Forest	11
4.6	Model F: Word2Vec + SVM with Feature-Level Resampling	11
5	Results Comparison & Visualization	12
5.1	Overall Performance: The Imbalance Illusion	12
5.2	Mid-Class Performance	13
5.3	Baseline Optimization: Pushing the Limits of BoW	15
5.4	Possible Improvements	15
5.4.1	Generative Large Language Model Integration	15
5.4.2	Aspect-Based Sentiment Analysis	16
6	Business Insights	16
7	Conclusion	17

1 Exploratory Data Analysis (EDA)

Before performing any data cleaning or modeling, a comprehensive audit was conducted on the raw dataset to identify structural issues, class imbalances, and text quality concerns. The dataset initially consists of 2,675 user reviews and 13 features.

1.1 Data Granularity

The dataset has a granularity of one row per user review. Each record represents an individual feedback instance submitted by a user at a specific point in time. This granularity allows for analysis at the review level, including temporal trends, version-specific feedback patterns, and user satisfaction distributions.

1.2 Dataset Overview & Missing Values

Initial inspection revealed that two columns, `language_code` and `country_code`, contained constant values ('en' and 'in', respectively) across all 2,675 rows. Since they provide no variance, they offer no discriminative power for modeling.

Furthermore, a missing value analysis identified severe data sparsity in several metadata fields. Figure 1 illustrates the percentage of missing data across the affected columns.

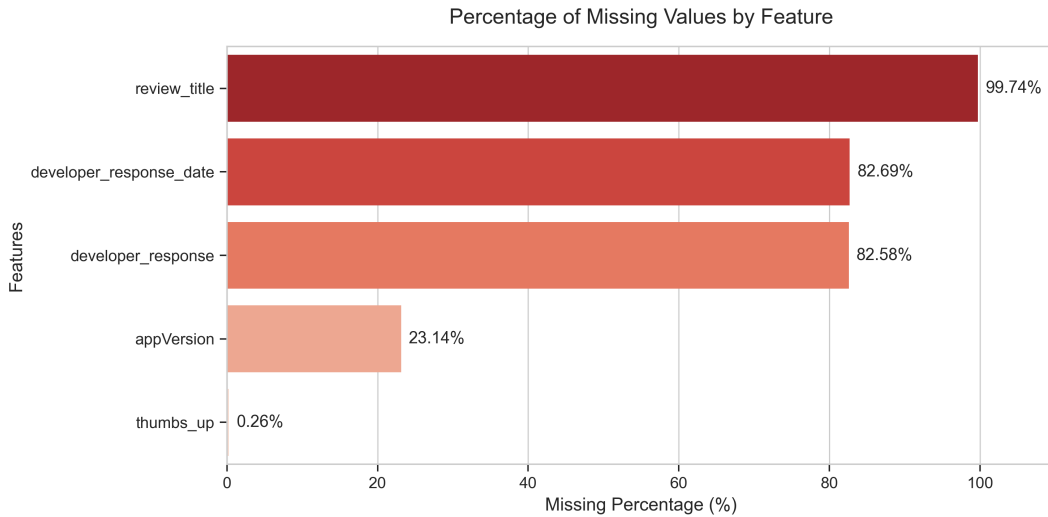


Figure 1: Percentage of missing values by feature in the raw dataset

The `review_title` feature is effectively empty, with a 99.74% missing rate. The developer response fields (`developer_response` and `developer_response_date`) are missing in over 82% of the records. The `appVersion` feature is missing in 23.14% of the instances. These findings directly inform the subsequent feature selection and imputation strategies during the data cleaning phase.

1.3 Target Variable: Rating Distribution

The primary objective of this project is to classify user sentiment based on the `rating` field. Understanding its distribution is critical, as it directly impacts model training. Figure 2 presents the exact count for each rating from 1 to 5 stars.

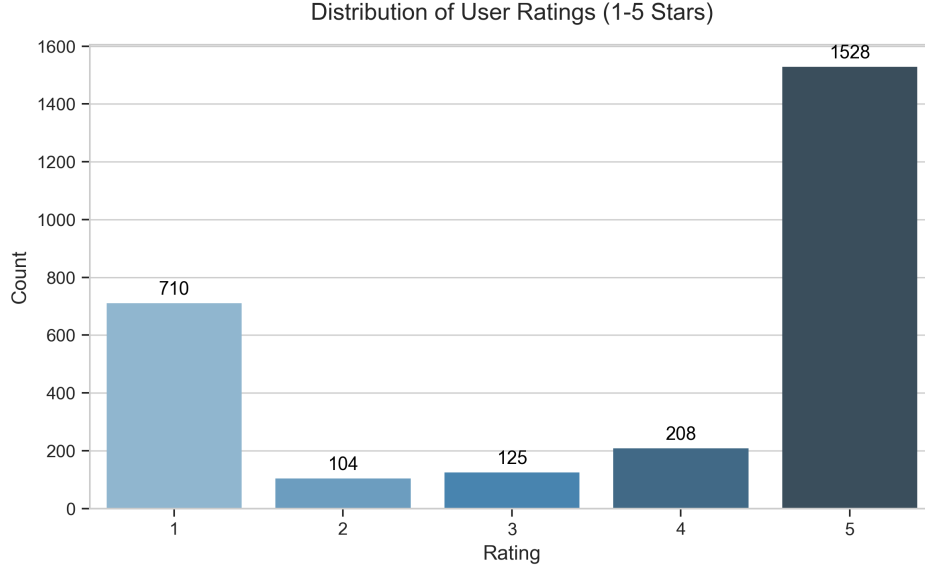


Figure 2: Distribution of user ratings (1-5 Stars)

The dataset exhibits an extreme class imbalance. The 5-star rating is the overwhelming majority class, accounting for 57.12% (1,528 reviews) of the dataset. The 1-star rating is the second largest at 26.54% (710 reviews). The middle ratings (2, 3, and 4 stars) are severely underrepresented, creating an imbalance ratio of 14.69 between the largest and smallest classes.

Without intervention, any machine learning classifier trained on this distribution would heavily bias towards 5-star predictions, failing to properly capture the semantic nuances of the minority middle ratings.

1.4 Text Characteristics & Quality Diagnostics

Given that the core modeling relies on Natural Language Processing (NLP), the quality and length of the `review_description` field were thoroughly audited. Figure 3 displays the distribution of review lengths measured by word count.

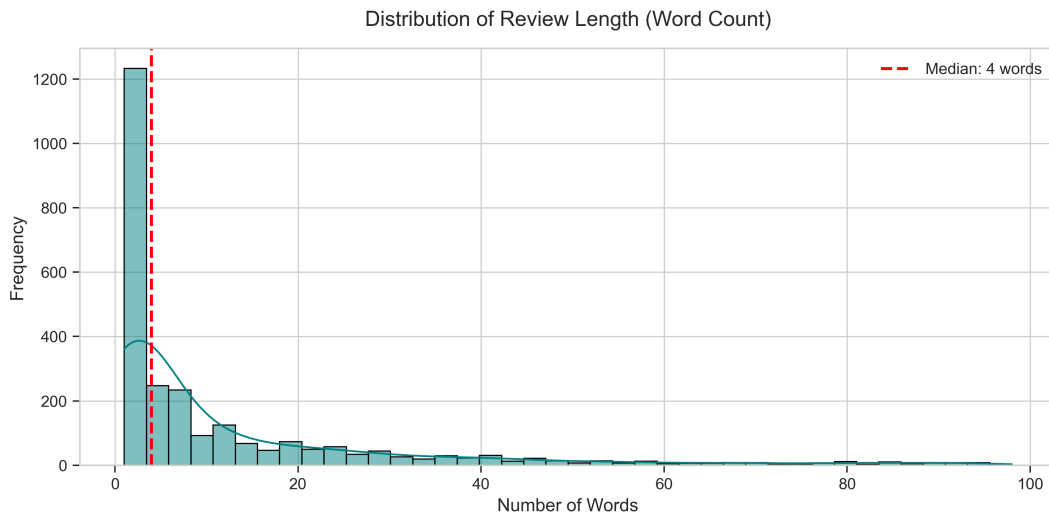


Figure 3: Distribution of review length (Word Count)

The text length is highly right-skewed. The median review length is extremely short, con-

taining only 4 words. In fact, 620 reviews consist of 1 word or fewer, relying heavily on brief, highly subjective terms like “Good” or “Nice”. Conversely, there are 286 long-text outliers extending up to nearly 100 words, which typically represent detailed user complaints.

Additionally, language risk checks detected that 176 rows (6.58%) contain non-ASCII characters, and 49 rows have an abnormally low ASCII-to-English letter ratio (< 0.5). The presence of non-English strings, typos, and extremely short sequences significantly increases the difficulty of feature extraction, justifying the need for robust tokenization and word embedding techniques (such as n-gram FastText and deep contextual BERT) in the later phases.

1.5 Outliers

1.5.1 Thumbs-Up Outliers

To identify exceptionally high-engagement reviews, thumbs-up outliers were examined using the 99th percentile as a threshold. Reviews with more than 13 thumbs-up votes were treated as highly viral reviews.

Table 1: Highly Viral Reviews (Top Thumbs-Up Counts)

Review Excerpt	Thumbs Up	Rating
This app is heavy and resource intensive. It drains the phone’s battery quickly and uses a lot of data...	606	1
This app is good but sometimes the drivers are so annoying especially the auto drivers. Few days ago...	752	1
So hard to find cabs. Every driver asks for the destination even if before heading toward the pickup...	221	1
This used to be a great app and now all the drivers purposely can trips to drive up pricing...	145	1
The latest update of the app really sucks, it’s really difficult now to get an uber to pick you up...	160	1

A total of 26 reviews exceeded the threshold. Most of these highly endorsed reviews are 1-star complaints that discuss concrete issues such as battery drain, driver behavior, pricing problems, and update-related failures. This suggests that users strongly value specific and actionable negative feedback.

1.5.2 Review-Length Outliers

A z-score method was used to identify unusually long reviews. Figure 4 shows the review-length outliers highlighted in red.

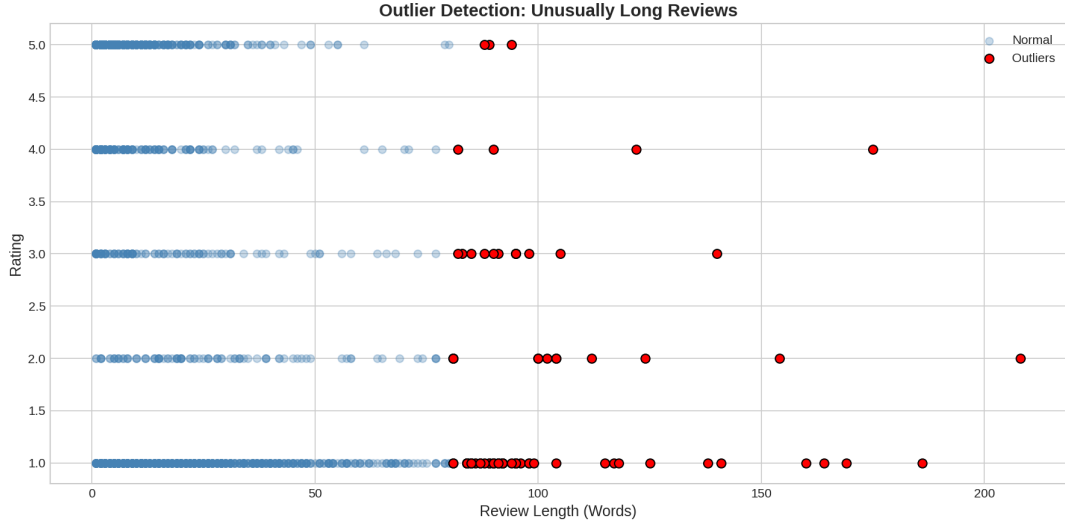


Figure 4: Scatter plot of review length vs rating with outliers highlighted (red points)

Unusually long reviews appear across all rating levels, but they are more concentrated among low ratings, especially 1-star reviews. This supports the earlier observation that dissatisfied users are more likely to provide detailed explanations.

2 Data Cleaning and Preprocessing

Following the initial audit, a systematic preprocessing pipeline was implemented to transform the noisy, imbalanced raw data into a robust training set. This phase focuses on feature selection, noise reduction, semantic consolidation, and class balancing.

2.1 Feature Selection and Missing Value Handling

To reduce dimensionality and remove uninformative data, eight metadata columns with near-zero variance or extreme missing rates (e.g., `review_title`, `language_code`, `country_code`) were dropped. For the retained features, missing values were systematically imputed: `thumbs_up` missing values were filled with 0 (assuming no user votes), and missing `appVersion` entries were filled with an “unknown” placeholder to maintain data integrity without discarding valuable text samples.

2.2 Text Normalization and Strict Data Filtering

To ensure the quality of textual features, all reviews were converted to lowercase and multiple whitespaces were collapsed. Subsequently, we applied a “Strict Filter” logic involving the following steps:

- **Noise Removal:** Reviews consisting only of punctuation, whitespace, or repeated character patterns (e.g., “goood”) were removed.
- **Language Filtering:** Although the metadata labeled all reviews as English, our EDA identified significant multilingual noise. We enforced a minimum ASCII letter ratio of 0.60 and filtered out reviews containing suspicious encodings (e.g., “Ã”, “â”).
- **Deduplication:** 619 duplicate reviews were removed to prevent the model from overfitting on frequent but uninformative templates.

2.3 Sentiment Label Mapping: From 5-Star to 3-Class

A critical decision in our pipeline was the remapping of the 5-star rating system into three sentiment categories: *Low* (0), *Mid* (1), and *High* (2).

Table 2: Label Mapping Schema

Original Rating	Sentiment Class	Label ID
1-star, 2-star	Low (Negative)	0
3-star	Mid (Neutral)	1
4-star, 5-star	High (Positive)	2

The rationale for this consolidation is twofold:

1. **Semantic Redundancy:** EDA revealed that reviews for 4 and 5 stars often share identical semantics, typically consisting of brief positive terms like "good" or "nice." Similarly, 1 and 2-star reviews frequently overlap in their descriptions of service failure.
2. **Subjectivity Mitigation:** Distinguishing between adjacent ratings (e.g., 4 vs. 5) is often highly subjective for users. Mapping them to a single "High" class reduces label noise and creates more distinct boundaries for the classifiers.

2.4 Data Splitting and Balancing via NLLB Back-Translation

Before addressing the severe class imbalance, the cleaned dataset was split into a **Training Set (80%)** and a **Test Set (20%)** using stratified sampling. Splitting the data *before* applying any augmentation is a critical step to prevent data leakage and ensure the test set remains completely unseen and authentic.

Despite the label mapping, the *Mid* class remained severely underrepresented in the training set. To address this, we implemented a data augmentation strategy strictly on the training set using the **NLLB-200** distilled model from Meta [7].

Back-Translation Pipeline: For the minority classes (*Low* and *Mid*), we performed a round-trip translation: **English** → **Simplified Chinese** → **English**.

By translating through an intermediate language, we generated 2,480 high-quality synthetic samples. This technique introduces lexical and syntactic diversity (using different synonyms or sentence structures) while strictly preserving the original sentiment meaning.

2.5 Final Dataset Composition

As a result of the NLLB augmentation, the training dataset was transformed from an imbalanced collection into a perfectly balanced training set. The final augmented training set contains **3,927 rows** with a uniform **1:1:1 ratio** (1,309 samples per class) across all three sentiment classes. This balanced foundation eliminates majority-class bias and ensures the models can effectively learn the specific features of neutral and negative feedback, while the isolated test set guarantees a fair evaluation environment without any data leakage.

3 Phase 3i: Baseline Model Analysis

3.1 Objective and Data Preparation

The baseline model was formulated as a 3-class sentiment classification task: **low** (1–2 stars), **mid** (3 stars), and **high** (4–5 stars). The modeling pipeline leverages two specific datasets derived from the preprocessing phase:

- `data_train_ready.csv`: 3,927 rows (a perfectly balanced and NLLB-augmented training set).
- `data_test_ready.csv`: 501 rows (a naturally imbalanced held-out test set).

Notably, the *mid* class constitutes only 4.79% (24 samples) of the test set, which necessitates the use of **macro-averaged metrics** to fairly evaluate minority class performance.

3.2 Methodology: TF-IDF + Logistic Regression

To establish a reliable traditional machine learning benchmark, we implemented a pipeline consisting of a Bag-of-Words (BoW) feature extractor and a linear classifier:

- **Feature Extraction (TF-IDF)**: The `review_description` was vectorized using `TfidfVectorizer`. To capture local context while managing memory efficiency, the model was configured to extract unigrams and bigrams (`ngram_range=(1,2)`), remove English stopwords, filter extremely rare terms (`min_df=2`), and cap the vocabulary at 20,000 features.
- **Analytics Classifier (Logistic Regression)**: The classifier was optimized using the `lbfgs` solver with `max_iter=2000`. Crucially, to counteract the model’s natural bias toward the dominant *high* class, the `class_weight="balanced"` parameter was applied to dynamically penalize the misclassification of minority classes during training.

3.3 Baseline Performance

Evaluated on the held-out test set, the baseline achieved an overall accuracy of **80.64%** and a macro F1-score of **0.5671**. The detailed per-class metrics and the prediction distribution are presented in Table 3 and Table 4.

Table 3: Classification Report of the Baseline Model

Class	Precision	Recall	F1-score	Support
low	0.6715	0.9267	0.7787	150
mid	0.0714	0.0417	0.0526	24
high	0.9429	0.8073	0.8699	327
macro avg	0.5619	0.5919	0.5671	501
weighted avg	0.8199	0.8064	0.8034	501

Table 4: Confusion Matrix of the Baseline Model

True Class \ Predicted	Pred low	Pred mid	Pred high
low	139	3	8
mid	15	1	8
high	53	10	264

3.4 Limitations and Error Analysis

While the 80.64% accuracy superficially appears strong, the confusion matrix exposes the fundamental limitations of the traditional TF-IDF approach, justifying the need for advanced models:

- **Failure on Neutral Sentiment:** Despite the use of balanced class weights, the baseline correctly identified only 1 out of 24 *mid* reviews (Recall: 4.17%). The majority of neutral reviews (15 cases) were entirely misclassified as *low*.
- **Lack of Semantic Context:** A significant volume of *high* reviews (53 cases) were incorrectly classified as *low*. This highlights a classic shortcoming of TF-IDF: it readily detects isolated negative words (e.g., "not") but fundamentally fails to process contextual negations or subtle semantic shifts (e.g., "not bad at all").

These findings confirm that frequency-based token matching and linear classification are insufficient for the colloquial and ambiguous nature of Uber customer reviews. This establishes a clear mandate for **Phase 3ii**, where we introduce advanced contextual embeddings (FastText and BERT) to capture deep semantic meaning and rescue minority class performance.

3.5 Strategic Transition: From Traditional to Contextual

The preliminary analysis above reveals that the default Baseline configuration struggles to differentiate ambiguous neutral sentiments from polar extremes. While these findings establish a mandate to immediately transition to advanced architectures in Phase 3ii (Section 4), they also raise an important question: can the traditional BoW paradigm be "rescued" using techniques from modern modeling?

To maintain a rigorous experimental flow, we will first explore six advanced embedding-based models in the following section. Subsequently, in **Baseline Optimization: Pushing the Limits of BoW (Section 5.3)**, we will return to the baseline to perform a "Post-hoc Optimization" trial. By applying insights gained from advanced modeling—such as Optuna tuning [2], sublinear scaling, and robust 5-fold cross-validation—we will establish the absolute performance ceiling of TF-IDF. This approach ensures our final comparisons are truly equitable and helps distinguish between hyperparameter issues and fundamental algorithmic limitations.

4 Phase 3ii: Advanced Modeling

4.1 Model A: GloVe + BiLSTM

This section introduces the implementation of a **GloVe** pre-trained embedding model [4] combined with a **Bidirectional LSTM** (BiLSTM) as the analytics classifier, representing the only sequential modelling approach in this study.

- **Embedding Generation (GloVe):** The pre-trained `glove.6B.100d` vectors were loaded to initialise an `nn.Embedding` layer of dimension 100. A vocabulary was constructed from the training subset only. Words found in the GloVe vocabulary were assigned their pre-trained vectors directly; out-of-vocabulary words were initialised from a normal distribution ($\mu = 0$, $\sigma = 0.1$). The embedding weights were frozen throughout training (`requires_grad=False`), ensuring GloVe serves purely as the embedding stage. Each review was tokenised by lowercasing and removing non-alphanumeric characters, then padded or truncated to a maximum of 64 tokens.
- **Analytics Classifier (BiLSTM):** A two-layer bidirectional LSTM with hidden dimension 128 per direction (256 combined) was deployed as the back-end analytic layer. The sentence representation was formed by concatenating the final forward and backward hidden states, yielding a 256-dimensional vector passed through a linear classification layer. Dropout (rate = 0.4) was applied to both input embeddings and the concatenated hidden states for regularisation. Gradient clipping (`max_norm=1.0`) was applied during training to prevent exploding gradients.

- **Class Imbalance Handling:** A weighted `CrossEntropyLoss` was used during training, with the *mid* class weight automatically computed as $w_{mid} = \frac{\max(\text{class counts})}{\text{mid count}}$ and weights of 1.0 assigned to the *low* and *high* classes.
- **Training Configuration:** The model was optimised using Adam with an initial learning rate of 10^{-3} . `ReduceLROnPlateau` reduced the learning rate by a factor of 0.5 when validation Macro F1 showed no improvement for 3 consecutive epochs. Training ran for 20 epochs with a batch size of 64, with the checkpoint achieving the highest validation Macro F1 retained for final evaluation.

4.2 Model B: BERT + XGBoost

This section details the implementation of a decoupled architecture utilizing **BERT** embeddings [1] as the deep contextual feature extractor and **XGBoost** [6] as the analytics classifier.

- **Input Processing:** The raw review texts were processed using the `bert-base-uncased` tokenizer, with sequences padded and truncated to a maximum length of 128 tokens to ensure uniform input dimensions.
- **Embedding Generation (BERT):** The pre-trained `bert-base-uncased` model was deployed in evaluation mode (`model.eval()`) as a frozen feature extractor. The 768-dimensional [CLS] token vector from the final hidden layer was extracted as the sentence-level representation, capturing the full contextual semantics of each review.
- **Analytics Classifier (XGBoost):** An `XGBClassifier` with `objective= multi:softmax` and `tree_method=hist` was deployed as the back-end analytic layer. To address class imbalance, a sample weighting strategy was adopted, with the *mid* class weight incorporated as a tunable hyperparameter in the Optuna [2] search space (range: 1–18), allowing the optimiser to automatically determine the most effective penalty for the minority class.
- **Hyperparameter Optimisation:** Bayesian optimisation was performed using Optuna [2] over 50 trials. To prevent excessive *mid* class weighting from degrading *high* class performance, a composite validation objective was adopted: $0.55 \times \text{Macro F1} + 0.45 \times \text{F1}_{high}$. The search space covered key XGBoost parameters including number of estimators (150–600), maximum depth (4–12), learning rate (0.02–0.25), subsample ratio (0.65–1.0), column subsampling ratio (0.65–1.0), L1 and L2 regularisation, and maximum delta step (0–5).
- **Feature Interpretability:** Post-training, XGBoost feature importance scores were extracted and visualised for the top 15 most discriminative BERT semantic dimensions, providing a traceable link between the high-dimensional embedding space and the final classification decisions.

4.3 Model C: BERT + Random Forest

This section details the implementation of a decoupled architecture utilizing **BERT** embeddings [1] as the deep semantic feature extractor and a **Random Forest** as the analytics classifier. The methodology is strictly aligned with the experimental code as follows:

- **Input Processing:** The raw review texts were processed using the `bert-base-uncased` tokenizer. To handle the varying lengths of mobile app reviews while maintaining computational efficiency, the tokenized sequences were padded and truncated to a maximum length of 128 tokens, ensuring uniform input dimensions for the transformer model.

- **Label Encoding:** The dataset was structured into a three-class classification task, with sentiment polarities encoded as *low* (Class 0), *mid* (Class 1), and *high* (Class 2).
- **Embedding Generation (BERT):** A pre-trained `bert-base-uncased` model was deployed in evaluation mode (`model.eval()`) to serve as a frozen feature extractor. Instead of traditional token averaging, the 768-dimensional output vector corresponding to the special [CLS] token from the final hidden state was extracted. This vector acts as an aggregated, context-aware semantic representation of the entire review sequence.
- **Analytics Classifier (Random Forest):** A `RandomForestClassifier` was deployed as the back-end analytic layer. To combat the severe class imbalance identified in the dataset, a highly aggressive **Manual Cost-Sensitive Learning** strategy was integrated. Custom class weights were dynamically injected into the pipeline (specifically utilizing a heavy penalty such as $\{0 : 1, 1 : 50, 2 : 1\}$), forcibly increasing the model’s sensitivity to the minority *mid* class to rescue its recall rate.
- **Hyperparameter Optimisation:** Bayesian optimisation was performed using the Optuna framework [2] to systematically search the hyperparameter space. The objective function was specifically set to maximize the Macro F1-score on a held-out validation set to ensure minority class representation. The search space included the number of estimators (100–500), maximum tree depth (10–50), minimum samples split (2–10), minimum samples leaf (1–5), and the maximum features strategy.
- **Feature Interpretability:** Post-optimization, the Gini feature importance was extracted from the optimal Random Forest model. This enabled the visualization of the top 15 most discriminative BERT semantic dimensions, establishing a traceable link between the high-dimensional latent space and the final classification logic.

4.4 Model D: FastText + SVM

This section details the implementation of a decoupled architecture utilizing **FastText** embeddings [5] as the feature extractor and a **Linear SVM** as the analytics classifier. The methodology is strictly aligned with the experimental code as follows:

- **Input Tokenization:** The text data was standardized through lowercasing and simple whitespace-based tokenization. FastText was deliberately selected for its character n-gram mechanics, which inherently process sub-word information, thereby offering robust feature extraction against typographical errors commonly found in application reviews.
- **Label Encoding:** The dataset was structured into a three-class classification task, with sentiment polarities encoded as *low* (Class 0), *mid* (Class 1), and *high* (Class 2).
- **Embedding Generation (FastText):** A custom FastText model was trained from scratch on the training subset using the Skip-gram architecture (`sg=1`). The hyperparameters were strictly defined: a vector size of 100, a context window of 5, a minimum word count of 2, and a training duration of 20 epochs. To transform dynamic-length token lists into fixed-length inputs, a mean-pooling strategy was applied, averaging the valid token vectors to produce a 100-dimensional sentence embedding.
- **Feature Normalization:** Prior to classification, the aggregated sentence embeddings were fitted and transformed using `StandardScaler`. This ensured the feature space maintained zero mean and unit variance, a mathematical prerequisite for the margin-based optimization in SVM.

- **Analytics Classifier (Linear SVM):** The `LinearSVC` algorithm was deployed as the back-end classifier with `max_iter=10000` to guarantee convergence. To facilitate cost-sensitive learning, a custom class-weight dictionary (`custom_weights`) was integrated into the pipeline. For this benchmark execution, a uniform weighting scheme (`{0 : 1, 1 : 1, 2 : 1}`) was explicitly assigned, establishing a baseline to evaluate the raw discriminative power of the FastText embeddings without algorithmic bias.
- **Hyperparameter Optimisation:** A deterministic grid search was conducted to find the optimal regularization parameter (C) for the Linear SVM. Evaluated across candidate values of `{0.01, 0.1, 1.0, 5.0}`, the optimization objective was to maximize the validation Macro F1-score, striking the optimal balance between margin maximization and classification accuracy on the validation subset.

4.5 Model E: FastText + Random Forest

This section introduces the implementation of a **FastText** word embedding model [5] with a **Random Forest** classifier. The modeling architecture is as follows:

- **Embedding Model (FastText):** A custom FastText model was trained on the tokenized review description data. In the implementation, each review was first lowercased, cleaned by removing non-alphanumeric characters, and split into tokens. FastText was then trained on the training corpus to produce 100-dimensional word vectors. To obtain a fixed-length representation for each review, the word vectors of all valid tokens were averaged into a single 100-dimensional sentence vector.
- **Analytics Classifier (Random Forest):** The sentence vectors were standardized using `StandardScaler` and then used as the input features of a `RandomForestClassifier`. The classifier was trained to map the dense FastText-based sentence representations to the target sentiment classes.

4.6 Model F: Word2Vec + SVM with Feature-Level Resampling

This section introduces the implementation of a **Word2Vec** embedding model [3] with a **Linear SVM** classifier, combined with feature-level resampling to address class imbalance. The modeling architecture is as follows:

- **Input Construction:** The raw review texts were first cleaned by removing invalid or empty entries. In addition to the review description, available metadata fields (e.g., app version, thumbs-up count, and review date) were concatenated with the text to form the final input sequence for each sample.
- **Label Mapping:** The original rating scores were mapped into three classes: ratings of 1 were labeled as *low*, ratings of 2 and 3 as *mid*, and ratings of 4 and 5 as *high*, forming a three-class classification task.
- **Embedding Model (Word2Vec):** A custom Word2Vec model was trained on the training corpus to learn dense word representations based on contextual co-occurrence patterns. Each token was mapped into a 100-dimensional vector space. To obtain fixed-length representations, the vectors of all valid tokens in a review were averaged to produce a sentence-level embedding.
- **Feature Transformation:** The resulting sentence embeddings were standardized using `StandardScaler`. PCA was optionally applied to transform the feature space while preserving most of the variance.

- **Feature-Level Resampling (Borderline-SMOTE):** To address class imbalance, Borderline-SMOTE [8] was applied to the training set in the feature space. This method generates synthetic samples near the decision boundary, balancing the class distribution without modifying the original text data.
- **Analytics Classifier (Linear SVM):** The processed features were used to train a LinearSVC classifier. The regularization parameter C was selected from a predefined set based on validation macro F1-score, ensuring balanced performance across all classes.

5 Results Comparison & Visualization

This section compares the performance of 7 models including 1 baseline model and 6 advanced models. Comparison metrics consist of predicting accuracy, macro F1 score for each classes.

5.1 Overall Performance: The Imbalance Illusion

Table 5 and Figure 5 present a comprehensive comparison of the overall Accuracy and Macro F1 scores across the Baseline model and the six advanced modeling approaches. A preliminary observation reveals a striking discrepancy: while all models easily achieve a high accuracy ranging from 73.8% to 83.8%, their Macro F1 scores stall significantly, hovering between 56.0% and 62.0%.

Table 5: Comprehensive Performance Summary of Baseline and Advanced Models

Model	Architecture (Embedding + Classifier)	Accuracy	Macro F1	Weighted F1	Imbalance Strategy
Baseline	TF-IDF + Logistic Regression	0.8060	0.5670	-	Default / Undefined
Model A	GloVe + BiLSTM	0.8244	0.5927	0.8205	Weighted CrossEntropyLoss
Model B	BERT + XGBoost	0.8363	0.6205	0.8307	Sample Weighting via Optuna
Model C	BERT + Random Forest	0.8283	0.5793	0.8185	Manual Cost-Sensitive Weights
Model D	FastText + SVM	0.7385	0.5700	0.7700	Uniform Class Weights
Model E	FastText + Random Forest	0.8383	0.6000	0.8300	Optuna Hyperparameter Tuning
Model F	Word2Vec + LinearSVC	0.7447	0.5914	0.7736	Borderline-SMOTE (Resampling)

This significant discrepancy is evidence of the Imbalance Illusion. Because the evaluation set is dominated by the majority classes, a classifier can obtain a very high accuracy simply by predicting the Low and High classes correctly, even if it entirely fails to recognize the minority Mid class. The Baseline model (TF-IDF + Logistic Regression) exemplifies this issue, yielding a seemingly strong accuracy of 80.6% but a low Macro F1 of 56.7%, which indicates poor class-level balance.

When comparing the advanced models, Model E (FastText + Random Forest) achieved the highest overall accuracy at 83.8%. However, it is Model B (BERT + XGBoost) that provided the best structurally balanced performance, securing the highest Macro F1 score of 62.1% while maintaining a highly competitive accuracy of 83.6%. This suggests that the combination of deep contextual embeddings from BERT [1], the gradient boosting mechanics of XGBoost [6], and optimized sample weighting effectively mitigated some of the majority-class bias without sacrificing overall correctness.

Conversely, Model D (FastText + SVM) and Model F (Word2Vec + Borderline-SMOTE + LinearSVC) show a distinct behavior pattern. Their overall accuracy dropped significantly to 73.9% and 74.5%, respectively, yet their Macro F1 scores (57.0% and 59.1%) remained competitive with or even exceeded the Baseline. This drop in accuracy is actually a positive signal of trade-off: these models, especially Model F with its Borderline-SMOTE [8] feature resampling, shifted their decision boundaries to capture the harder minority samples, inevitably sacrificing some precision on the dominant classes to achieve a more equitable learning outcome.

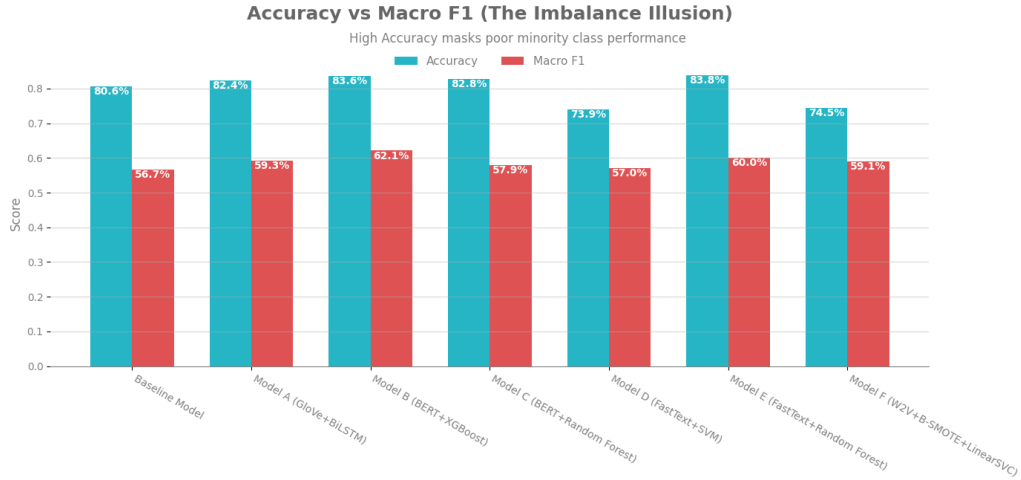


Figure 5: Accuracy and Macro F1 comparison across the baseline model and six advanced models.

5.2 Mid-Class Performance

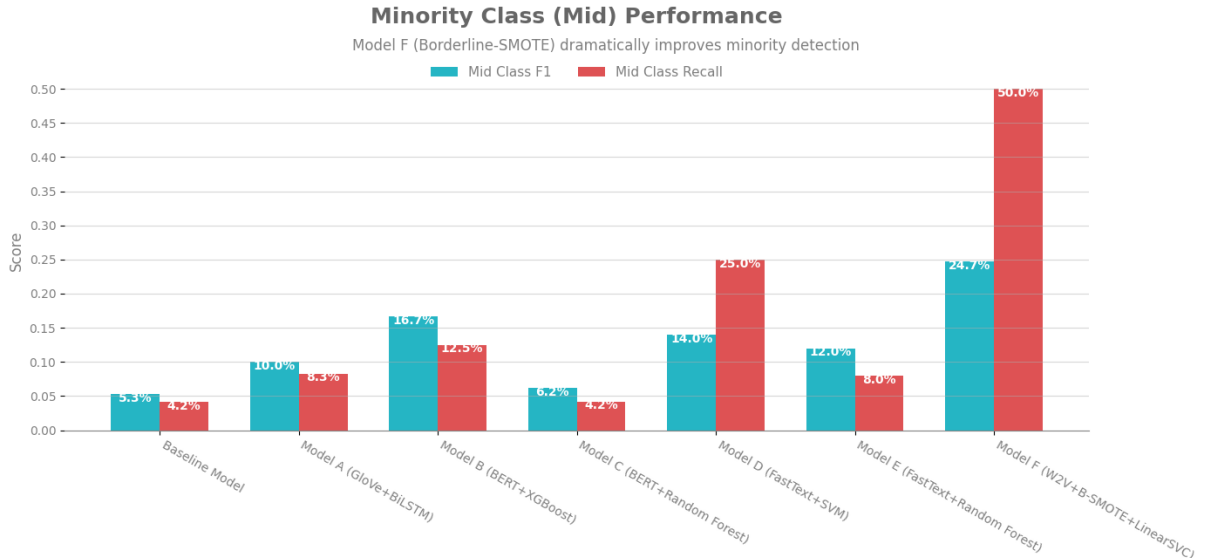


Figure 6: Mid-class recall and F1 comparison across the baseline model and six advanced models.

The mid class sentiments prediction is one of the main bottleneck of the project. In the baseline model, only 1 out of 24 mid-class reviews was correctly identified, with recall score of 0.0417 and an F1-score of only 0.0526. Model A and Model B showed only a marginal improvement over the baseline on the mid class, while Model E also remained weak despite using sequential deep learning architecture. This indicates that better semantic representation alone does not automatically solve the minority-class problem.

Among the models under the common 501-sample test setting, Model D provided the strongest mid-class balanced result, reaching a mid-class F1-score of 0.1667. Model C achieved a higher mid recall of 0.2500, but its precision was only 0.0900 and its overall accuracy dropped to 0.7385, which means it captured more mid reviews at the cost of many false positives. Model B provided a better compromise than the baseline, but its mid-class recall and F1-score were still limited.

Model F was the most notable result for minority detection, reaching a mid recall of 0.500

and a mid F1-score of 0.247 after introducing Borderline-SMOTE [8] in feature space. This shows that explicit imbalance handling can improve the detection of hard boundary cases in the mid class.

To trace exactly how the mid-class reviews were misclassified, we visualized the row-normalised confusion matrices for all seven models in Figure 7.

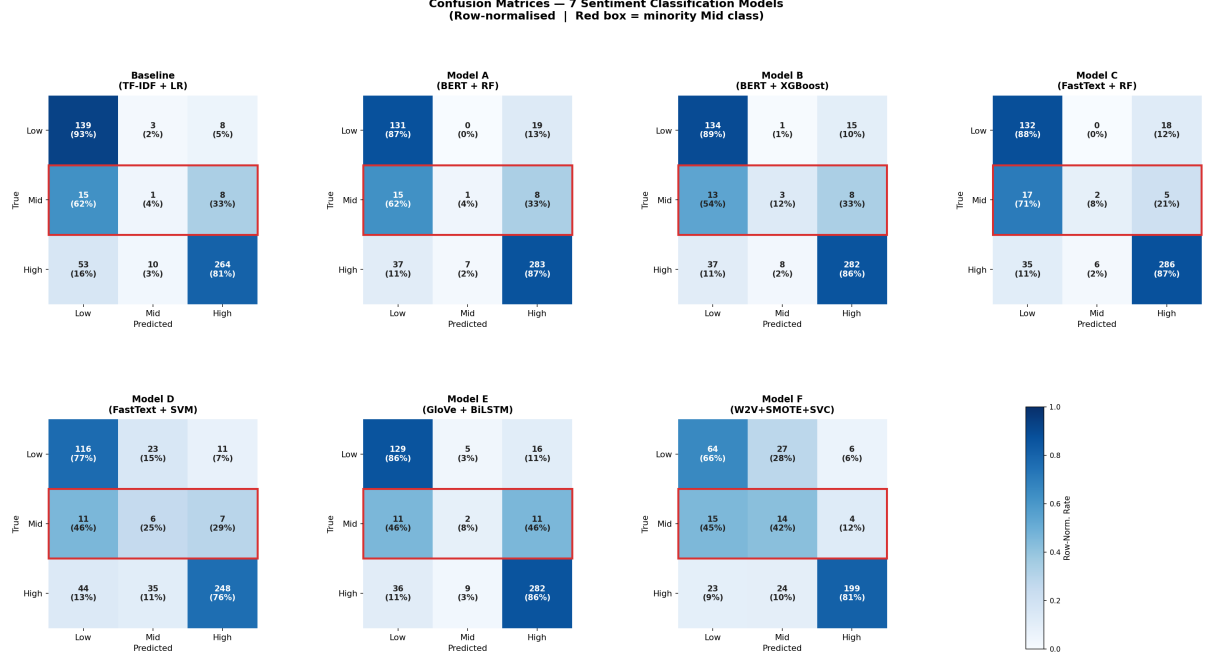


Figure 7: Row-normalised confusion matrices for the 7 models. The red boxes highlight the models’ predictions on the true minority Mid class. Darker blue indicates a higher proportion of predictions.

The heatmaps in Figure 7 provide clear visual evidence of the imbalance illusion. Looking at the red boxes (true mid class), we can see that for the Baseline model and Models A through E, the center cell (correct mid prediction) is extremely pale, while the left and right cells (misclassified as low or high) capture the vast majority of the mid-class samples. For instance, the Baseline model pushed 15 mid-class reviews to the low class and 8 to the high class, correctly identifying only 1.

Model F’s confusion matrix visually demonstrates its improved sensitivity to the minority class. The center cell in its red box is noticeably darker, indicating that Borderline-SMOTE [8] successfully pulled many mid-class reviews away from the dominant decision boundaries.

The persistent difficulty of the mid class, as illustrated by the error flows in the confusion matrices, can be explained by two factors. First, many mid reviews contain mixed opinions (e.g., a positive comment about the driver but a negative complaint about the app). This causes semantic neutralization, where positive and negative word vectors average out to near-zero, trapping the review within the decision boundaries of the low and high classes. Second, the mid class is severely underrepresented in the evaluation data, meaning the classifier tends to treat this minority class as noise.

Overall, if the goal is balanced global classification, BERT + XGBoost is currently the strongest model. If the goal is specifically to recover the minority mid class, the Borderline-SMOTE approach provides the most promising direction. Future work should consider integrating Generative LLM-based encoders to better capture contextual transitions (such as “but” and “although”), or adopting Aspect-Based Sentiment Analysis to separately evaluate multiple dimensions (like driver vs. app) within a single mixed-sentiment review.

5.3 Baseline Optimization: Pushing the Limits of BoW

To strictly verify whether the baseline’s performance bottleneck was due to sub-optimal parameters or an inherent algorithmic limitation, we conducted a rigorous optimization experiment on the TF-IDF + Logistic Regression pipeline. We implemented three targeted enhancements:

1. **Sublinear Term Frequency Scaling:** We enabled `sublinear_tf=True` to apply logarithmic scaling to term frequencies (replacing tf with $1 + \log(tf)$). This reduces the variance caused by drastically different review lengths, which is common in app store data.
2. **N-gram Expansion:** The contextual capture range was expanded to include both unigrams and bigrams (`ngram_range=(1,2)`), balancing local context retention with vocabulary sparsity.
3. **Robust Hyperparameter Tuning (5-Fold CV):** We utilized the **Optuna** framework [2] combined with 5-Fold Stratified Cross-Validation. The optimization objective was to maximize the validation Macro F1-score by searching for the optimal regularization strength (C) while maintaining a `class_weight="balanced"` penalty.

Optimization Results: The optimization converged at $C \approx 6.35$, achieving a validation Macro F1 of 0.9073. When evaluated on the held-out test set, the optimized baseline demonstrated significant gains: the overall **Accuracy increased to 84.83%** and the **Macro F1-score improved to 0.5938**.

Table 6: Classification Report and Confusion Matrix of the Optimized Baseline

Classification Report					Confusion Matrix			
Class	Precision	Recall	F1-score	Support		Pred low	Pred mid	Pred high
low	0.7706	0.8733	0.8187	150	True low	131	3	16
mid	0.0769	0.0417	0.0541	24	True mid	14	1	9
high	0.9214	0.8960	0.9085	327	True high	25	9	293
Accuracy		0.8483		501				
Macro Avg	0.5896	0.6037	0.5938	501				

Final Conclusion of the Baseline Phase: The optimization successfully refined the decision boundaries for the polar classes (Low and High). However, even at its absolute performance limit, the model correctly identified only 1 out of 24 *mid* reviews. This experiment provides a definitive conclusion: the failure on the *mid* class is not a tuning issue, but a **representational limit of the Bag-of-Words (BoW) paradigm**. Linear boundaries in a sparse frequency space lack the semantic depth to separate ambiguous neutral sentiments from noisy positive/negative signals. This comprehensively justifies the transition to the advanced contextual embeddings explored in this study.

5.4 Possible Improvements

While the advanced models achieved meaningful gains, they still struggle with the inherent linguistic complexity of neutral feedback. To overcome the representational and data bottlenecks identified in Section 5.2, we propose the following two structural enhancements for future implementation.

5.4.1 Generative Large Language Model Integration

The first proposed enhancement is to replace the current static or frozen embedding front-end with a generative Large Language Model (LLM). Current embedding approaches such as GloVe

[4], FastText [5], and Word2Vec [3] represent sentences through mean-pooled word vectors, meaning that a mixed review like “Nice driver, but the app crashed” produces a near-zero embedding vector because the positive and negative components cancel each other out. Even when using a frozen BERT encoder [1], the CLS token representation is extracted without fine-tuning on the sentiment task, which limits the model’s ability to capture subtle contextual transitions.

Generative LLMs, such as GPT-4 or LLaMA, are equipped with deep multi-head attention mechanisms that explicitly model the relationship between every token pair in a sequence. Crucially, these models are highly sensitive to discourse markers such as “but”, “although”, “however”, and “despite”, which are the key linguistic cues that define mixed-sentiment Mid-class reviews. By fine-tuning or prompt-engineering a generative LLM on the labelled Uber review corpus, the model would be capable of learning to weight these transition words more heavily, producing semantically richer review representations that are more separable in the decision boundary space. This would directly address the semantic neutralization bottleneck that caused every current model to systematically misclassify mid-class reviews into the Low or High categories.

5.4.2 Aspect-Based Sentiment Analysis

The second proposed enhancement is to adopt Aspect-Based Sentiment Analysis (ABSA) as a complementary classification framework. The current pipeline assigns a single sentiment label (Low, Mid, or High) to each review as a whole, which is fundamentally insufficient for multi-dimensional user feedback. In practice, a user reviewing an Uber experience may simultaneously rate the driver positively while rating the app negatively, or praise the price while criticizing the waiting time. When the positive and negative aspects are merged into a single classification target, the review is forced into the Mid category, even though it carries rich and actionable feedback.

ABSA decomposes each review into distinct opinion targets, for example: *Driver*, *App UI*, *Price*, *Waiting Time*, and *Customer Service*, and evaluates sentiment independently for each aspect. This would not only provide a more accurate and granular sentiment estimate for each individual review, but also enable a deeper level of business insight by identifying which specific dimensions of the Uber service are most frequently praised or criticised. For instance, if ABSA reveals that the Driver aspect consistently receives positive sentiment while the App aspect consistently receives negative sentiment, Uber’s product and operations teams can prioritise their resources accordingly. Ultimately, integrating ABSA would transform the current single-label classifier into a multi-dimensional decision support system, better aligned with the analytical complexity of real-world app-review data.

6 Business Insights

Based on the thorough analysis of user feedback patterns within the app store dataset, we have identified several critical areas for Uber to improve its product design and operational strategies. The following recommendations are derived directly from the structural and semantic characteristics of customer reviews.

- **Redefine the “Neutral” Customer Segment for Loss Prevention**

Our analysis finds that customer feedback shows a clear polarization phenomenon, with the majority of users rating either highly positive or negative. The minority of users who leave “neutral” (3-star) reviews are rarely actually neutral; their written content consistently conveys complex and intertwined emotions, mentioning the driver’s professional driving skills while also expressing complaints about the malfunction of the application. Uber

should stop treating neutral ratings as passive satisfaction. Instead, the customer retention team should give priority to communicating with this group, because the detailed and objective feedback they provide can offer the most practical solution paths for addressing specific problems, thereby preventing such issues from occurring before these users switch to competitors.

- **Focus on Long Comments User Feedback**

There is a significant pattern regarding text length: positive feedback is usually brief (frequently consisting of a single word like “good” or “nice”), while negative reviews are typically long, often detailing specific areas of dissatisfaction up to 100 words. Uber’s product and engineering teams should implement automated routing that prioritizes long-form text submissions for technical issue-resolution pipelines. Since lengthy reviews almost exclusively contain detailed bug reports, payment gateway failures, or usability complaints, focusing on these extensive texts will rapidly identify technical defects and accelerate the developing process of targeted application updates.

- **Develop Multi-angle Evaluation System**

Users tends to lower their overall rating due to a software disaster, even if the driver’s skill was excellent. This mixture of sentiments hinders the true root cause of user dissatisfaction. Uber are highly encouraged to develop its post-ride feedback interface to split the app experience from the driver experience. Instructing users with separate, distinct evaluation metrics will prevent excellent drivers from being unfairly penalized for technical outages and will provide the software team with precise, untainted data regarding app reliability.

7 Conclusion

This project developed and evaluated a decision support system for sentiment classification of Uber Play Store reviews. Through a systematic pipeline encompassing exploratory data analysis, rigorous preprocessing, baseline modeling, and six advanced architectures, we addressed the challenges of extreme class imbalance, short and noisy text, and the semantic complexity of neutral and mixed-sentiment reviews.

The exploratory analysis revealed a highly imbalanced rating distribution, with 5-star reviews constituting 57.12% of the dataset and 3-star reviews severely underrepresented. A strict preprocessing pipeline, including noise removal, language filtering, deduplication, and NLLB-based back-translation, successfully produced a balanced training set of 3,927 samples with a 1:1:1 class ratio, while preserving an authentic, imbalanced test set of 501 samples.

The baseline TF-IDF + Logistic Regression model achieved an accuracy of 80.64% but a macro F1-score of only 0.5671, correctly identifying just 1 out of 24 mid-class reviews. This confirmed the representational limitations of bag-of-words approaches for ambiguous neutral sentiment. Advanced models significantly improved balanced performance. Among them, **Model B (BERT + XGBoost)** delivered the highest macro F1-score of 0.6205 and an accuracy of 83.63%, demonstrating the benefit of deep contextual embeddings combined with gradient boosting and optimized sample weighting. **Model F (Word2Vec + Borderline-SMOTE + LinearSVC)** achieved the strongest mid-class recall (0.500) and mid-class F1 (0.247), showing that feature-level resampling effectively improves minority-class detection.

A post-hoc optimization of the baseline using Optuna and 5-fold cross-validation raised its macro F1 to 0.5938, but it still failed on the mid class (recall 0.0417), conclusively proving that the bottleneck is representational, not parametric. The persistent difficulty of the mid class stems from semantic neutralization in mixed-sentiment reviews and extreme test-set underrepresentation.

To overcome these limitations, we propose two future directions: (1) integrating generative large language models (e.g., GPT-4, LLaMA) to better capture discourse markers such as “but” and “although”, and (2) adopting aspect-based sentiment analysis to evaluate multiple dimensions (e.g., driver, app UI, waiting time) independently, transforming the current single-label classifier into a multidimensional decision support system.

References

- [1] Devlin, J. et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
- [2] Akiba, T. et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework.
- [3] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- [4] Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1532–1543).
- [5] Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5, 135–146.
- [6] Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785–794).
- [7] Costa-jussà, M. R., et al. (2022). No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*.
- [8] Han, H., Wang, W.-Y., & Mao, B.-H. (2005). Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning. In *International conference on intelligent computing* (pp. 878–887). Springer.